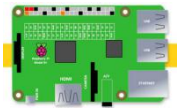
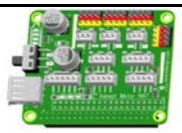
Lesson 7 How to Control RGB LED

In this lesson, we will learn how to control warm color LED with Raspberry Pi.

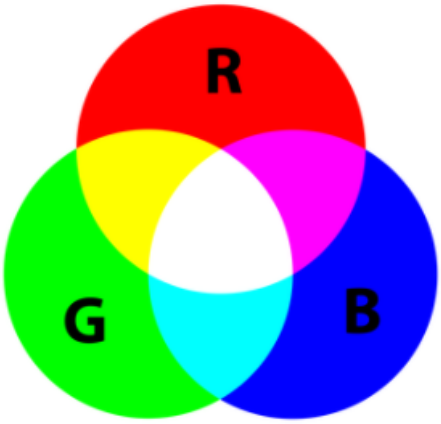
7.1 Components used in this course

Components	Quantity	Picture
Raspberry Pi	1	
Motor HAT	1	
RGB LED	1	

7.2 Introduction of RGB LED

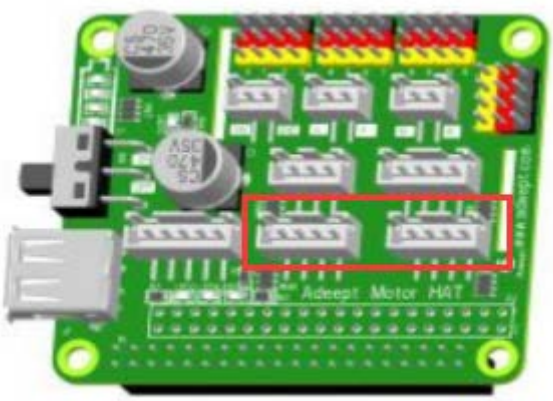
Using RGB LEDs, you can produce red, green and blue light, and by configuring the intensity of each LED, you can also produce other colors. For example, to produce pure blue light, you can set the red LED to the highest intensity, and the green and blue LEDs to the lowest intensity. For white light, replace with all three LEDs set to the highest intensity. To produce other colors, you can combine the three colors with different intensities. To adjust the intensity of each LED, you can use a PWM signal. Because the LEDs are very close to each other, what our eyes see is the result of the color combination, not the divided three colors.

The following chart. This is the simplest color mixing diagram.



7.3 Wiring diagram (Circuit diagram)

Use a 4pin cable to connect the RGB LED to RGB1 or RGB2 of Motor HAT.



7.4 Run the program of this course

1. Log in to your Raspberry Pi via SSH (the method to log in to the Raspberry Pi has been introduced in Lesson 3):

```
Linux raspberrypi 4.19.118-v7l+ #1311 SMP Mon Apr 27 14:26:42 BST 2020 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Aug 29 08:17:49 2020 from 192.168.3.208

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set
 a new password.

pi@raspberrypi:~ $
```

2. Run the command to enter the adeept_picarpro/server folder. This folder stores the sample code program for controlling the robot. Enter the following command and press Enter:

cd adeept_picar-b/server

```
pi@raspberrypi:~ $ cd adeept_picar-b/server/
pi@raspberrypi:~/adeept_picar-b/server $
```

3. Enter the command to view the contents of the current directory:

ls

```
pi@raspberrypi:~/adeept_picar-b/server $
pi@raspberrypi:~/adeept_picar-b/server $ ls
app.py            findline.py     Instruction.txt    __pycache__       servo.py
appserverAP.py    FPV.py          Kalman_filter.py   raspi-config.py   setup.py
appserver.py      FPVtest.py      LEDapp.py          RGB.py            speech.py
base_camera.py    functions.py    LED.py             robotLight.py     SR.py
camera_opencv.py  GUIfindline.py  move.py            RPIservo.py       switch.py
config.txt        GUImove.py      OLED.py            server.py         ultra.py
dist              info.py         PID.py             serverTest.py     webServer.py
pi@raspberrypi:~/adeept_picar-b/server $
```

4. RGB.py is a python program, you can run this program on the Raspberry Pi by directly typing the following commands:

sudo python3 RGB.py

```
pi@raspberrypi:~/adeept_picar-b/server $ sudo python3 RGB.py
```

5. After successfully running the program, you can observe that the RGB led will light up in different colors and turn off the RGB led after a short period of time. When you want to terminate the running program, you can press the shortcut key "Ctrl + C" on the keyboard.

7.5 Main code program

The complete code reference file RGB.py.

```
import RPi.GPIO as GPIO
import time
import threading
```

First import the library used to control the Raspberry Pi GPIO, and instantiate it as a GPIO while importing it. Import the time library for code delay.

```
left_R = 22
left_G = 23
left_B = 24

right_R = 10
right_G = 9
right_B = 25
```

The RGB LED is connected to the pin number of the Raspberry Pi, here we use BCM coding.

```
class RGBctrl(threading.Thread):
  def __init__(self):
    pass

  def setupTest(self, Rpin, Gpin, Bpin):
    global pins
    global pwmR, pwmG, pwmB
    pins = {"pin_R":Rpin, "Pin_G":Gpin, "pin_B":Bpin}
    GPIO.setmode(GPIO.BCM)
```

```
    for i in pins:
        GPIO.setup(pins[i], GPIO.OUT)     # Set the pin mode to output
        GPIO.output(pins[i], GPIO.LOW)     # Set the pin to low(0V) to off led

    pwmR = GPIO.PWM(pins['pin_R'], 50)     # Set Frequece to 50Hz
    pwmG = GPIO.PWM(pins['Pin_G'], 50)
    pwmB = GPIO.PWM(pins['pin_B'], 50)

    pwmR.start(0)                    # Initial duty Cyle = 0(leds off)
    pwmG.start(0)
    pwmB.start(0)

  def map(self, x, in_min, in_max, out_min, out_max):   # Map 0-255 to 0-100
    return (x - in_min) / (in_max - in_min) * (out_max - out_min) +out_min

  def off(self):
    for i  in pins:
        GPIO.output(pins[i], GPIO.LOW)
  def setColor(self, col):                    # Col is a 6-digit hexadecimal number. E.g: 0X112233
    R_val = (col & 0xff0000) >> 16          # Extract the value of the highest two digits. E.g: 11
    G_val = (col & 0x00ff00) >> 8
    B_val = (col & 0x0000ff) >> 0          # Extract the value of the lowest two digits.  E.g: 33

    R_val = self.map(R_val, 0, 255, 0, 100)        # Map 0-255 to 0-100.
    G_val = self.map(G_val, 0, 255, 0, 100)
    B_val = self.map(B_val, 0, 255, 0, 100)

    pwmR.ChangeDutyCycle(R_val)             # Change the duty cycle,change rhe color brightness.
    pwmG.ChangeDutyCycle(G_val)
    pwmB.ChangeDutyCycle(B_val)

  def destroy(self):
    pwmR.stop()                      # Turn off PWM.
    pwmG.stop()
    pwmB.stop()
    self.off()                       # Turn off all leds.
    GPIO.cleanup()                   # Reset GPIO status.
```

Define related functions.

```
if __name__ == "__main__":
  try:
    RGBCtrl_1 = RGBctrl()
```

```
    RGBCtrl_1.setupTest(left_R, left_G, left_B)     # Set RGB pin number.
    while True:
        RGBCtrl_1.setColor(0xff0000)
        time.sleep(1)
        RGBCtrl_1.setColor(0x00ff00)
        time.sleep(1)
except KeyboardInterrupt:
    RGBctrl().destroy()
```

Instantiate the function and cycle the RGB lights to different colors.